RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering

Academic Year: 2025 – 2026

MINI PROJECT REPORT

On

Moving Object Detection and Trajectory Tracking

Image Processing and Computer Vision Project

Submitted by

Deepak K

Roll No: 2116230701066

Section: [CSE]

Email: 2116230701066@rajalakshmi.edu.in

Date of Submission: April 2026

TABLE OF CONTENTS

TABLE OF CONTENTS…….2
ABSTRACT……..4
1. INTRODUCTION…..…….5
1.1 Background and Motivation…….....5
1.2 Problem Statement……...5
1.3 Objective……..5
1.4 Tools Used……....5
2. DATASET DESCRIPTION…….…..6
2.1 VIRAT Video Dataset……...6
2.2 Custom Campus Videos……...…6
3. PREPROCESSING METHODOLOGY……..……7
3.1 Preprocessing Visualizations……....7
4. FEATURE EXTRACTION & ALGORITHM INTERNALS…….…...8
5. ALGORITHM IMPLEMENTATION……..…9
5.1 ViBe Background Subtraction……..……9
5.2 Morphological Cleanup & Object Detection……..…..9
6. SYSTEM ARCHITECTURE…….…10
7. RESULTS AND EVALUATION…….....11
8. CONCLUSION…….....13
9. FUTURE WORK…….....13
10. REFERENCES……..……..13
APPENDIX A - SOURCE CODE……14
APPENDIX B - SAMPLE OUTPUT SCREENSHOTS…….25

LIST OF FIGURES AND TABLES

Figures

Figure 1: Before Gaussian denoise & CLAHE enhancement……...7
Figure 2: After Gaussian denoise & CLAHE enhancement…….7
Figure 3: ViBe background model sample visualization……..8
Figure 4: System Architecture Flowchart……...11
Figure 5: FPS Performance Over Time…...……12
Figure 6: Objects Detected Over Time……...13
Figure 7: Motion Intensity Over Time……13
Figure 8: Processing Time Per Frame…….13

Tables

Table 1 – Tools Used…….5
Table 2 – Dataset description including VIRAT and custom campus videos……...6
Table 3 – ViBe Algorithm Parameters……..9
Table 4 – System Architecture……10
Table 5 – Performance evaluation through frame-by-frame analysis……12

ABSTRACT

Modern video surveillance systems require highly accurate, robust, and real-time detection and tracking of moving objects such as pedestrians and vehicles. Traditional computer vision techniques often suffer from false positives due to illumination changes, shadows, and occlusions. This project presents a custom, academically-grounded pipeline that employs the **ViBe (Visual Background Extractor)** algorithm — a pixel-sample-based background subtraction method — combined with a centroid-based multi-object trajectory tracker, all packaged in a modern **PyQt5** graphical user interface.

The proposed system was developed and evaluated on the widely recognized **VIRAT Video Dataset** as well as custom-recorded campus surveillance footage. Through a combination of noise reduction, CLAHE contrast enhancement, morphological mask refinement, and real-time analytics visualization, the system achieved strong quantitative results. The final empirical evaluation yielded a **Precision of 0.9924, a Recall of 0.9290, an F1 Score of 0.9596**, and a **Tracking Accuracy of 0.9950**, confirming its scalability for automated CCTV monitoring applications.

1. INTRODUCTION

1.1 Background and Motivation

The evolution of computing power and artificial intelligence has revolutionized Computer Vision, particularly within the domains of intelligent video surveillance. Historically, detecting physical presence relied on static background subtraction, which crumbled under complex environmental variations like shifting shadows and illumination changes.

This project leverages the **ViBe (Visual Background Extractor)** algorithm — a lightweight, pixel-sample-based technique proposed by Barnich & Van Droogenbroeck (2011) — to deliver real-time foreground segmentation. Rather than relying on costly CNN inference, ViBe maintains a statistical model of each pixel's history, enabling it to process video at interactive frame rates on standard hardware.

1.2 Problem Statement

Traditional surveillance depends primarily on persistent human monitoring, an approach fraught with fatigue-induced errors and significant financial overhead. Algorithmic replacements frequently face ID-switching errors — where a tracked object passes behind an occlusion and emerges categorized as an entirely new entity. Furthermore, background-only methods like MOG2 lack semantic discrimination, producing noisy detections in complex scenes.

The critical problem is therefore engineering a highly resilient pipeline that combines accurate background subtraction, morphological noise cleanup, and temporal ID persistence, all while operating at real-time inference speeds compatible with standard video sources.

1.3 Objectives

- Construct a motion detection subsystem using the custom ViBe algorithm to isolate foreground pixels from static background.
- Apply a morphological post-processing pipeline (Erosion, Dilation, Closing) to remove noise and refine detected blobs.
- Integrate a centroid-based multi-object tracker for strict ID persistence across frames.
- Package all functionality inside an interactive PyQt5 GUI with tabbed video views and real-time analytics.

1.4 Tools Used

Tool	Role	Purpose
Python 3.10+	Core Execution Runtime	High-level data bindings, algorithm orchestration, and application entry point.
OpenCV 4.8+	Matrix Computations	Fast linear filtering, morphological operations, and graphic augmentations.
NumPy	Array Processing	Vectorized background model storage and pixel-level arithmetic.
ViBe (Custom)	Background Subtraction	Pixel-sample-based foreground segmentation without deep learning.
PyQt5	GUI Framework	Tabbed video panel, analytics dashboard, and threading for smooth UI.
Matplotlib	Charting	Embedded real-time FPS and performance charts in the analytics panel.
SciPy	Distance Computation	Centroid distance matrix for multi-object tracking association.

Table 1 – Tools Used

2. DATASET DESCRIPTION

2.1 VIRAT Video Dataset

The primary foundation for empirical training and automated evaluation within this project relies upon the **VIRAT Video Dataset**. Designed expressly to advance the field of event recognition and tracking, VIRAT features realistic surveillance-perspective camera angles, encompassing highly clustered human behaviors, vehicular movements, and complex spatial interactions.

The subset utilized includes 57 varied high-resolution MP4 clips representing hours of real-world occurrences. Frame rates oscillate dynamically but standard metrics average 30 FPS. Utilizing this dataset prevents the model from overfitting against 'perfect' scenarios. All frames are processed through the ViBe background model which initializes itself from the very first frame using spatial neighborhood sampling.

2.2 Custom Campus Videos

To validate domain-adaptation resilience, custom-recorded campus video clips were incorporated. These videos introduce unique elevation angles, variable lens distortion, and localized topologies that ensure the algorithm performs robustly beyond VIRAT's parameters.

Property	Details
VIRAT Clips	57 high-resolution MP4 surveillance videos
Resolution	640×480 (after preprocessing resize normalization)
Frame Rate	Average 30 FPS
Subjects	Pedestrians, cyclists, vehicles, mixed interactions
Campus Clips	Custom-recorded at variable elevation angles
Unique Challenges	Shadows, partial occlusions, illumination variance, crowded scenes

Table 2 – Dataset description including VIRAT and custom campus videos.

3. PREPROCESSING METHODOLOGY

Clean, standardized inputs are mathematically critical for consistent tracking performance. Raw camera feeds inherently contain Gaussian noise resulting from sensor heat, ISO limitations, and lossy MP4 compression protocols. The preprocessing pipeline explicitly rectifies these defects prior to ViBe inference.

Initially, videos are frame-extracted and routed through a resolution uniformizer (640×480). Subsequently, a classical **Gaussian convolutional blur** is applied (kernel: 5×5, σ=1.0). Following noise reduction, **Contrast Limited Adaptive Histogram Equalization (CLAHE)** is leveraged dynamically on localized tile grids (clipLimit=2.0, tileGridSize=8×8). Optionally, a **Median filter** can be toggled for additional salt-and-pepper noise suppression. The output is profoundly improved edge definition that stabilizes the ViBe background model.

3.1 Preprocessing Visualizations

Figure 1: Before Gaussian denoise & CLAHE enhancement.

Figure 2: After Gaussian denoise & CLAHE enhancement.

4. FEATURE EXTRACTION & ALGORITHM INTERNALS

Feature extraction in the ViBe framework operates at the pixel level rather than through high-dimensional neural embeddings. Each pixel maintains a **sample buffer of N=20 historical values** drawn from its own temporal and spatial neighborhood. This buffer acts as a compressed statistical signature of the background at each point in the image.

For segmentation, the algorithm computes the absolute distance between the current pixel value and all stored samples. A pixel is classified as **background** if at least **min_matches=2** samples fall within radius **R=20**. Otherwise, it is classified as **foreground**. This sample-counting approach is inherently robust to gradual illumination changes and avoids the Gaussian distribution assumptions of MOG2.

To visualize the high-dimensional implicit feature space of the detector, the ViBe background model's per-pixel sample distributions were analyzed. The multi-dimensional feature arrays encode temporal statistics at each spatial location, allowing the algorithm to mathematically distinguish dynamic foreground entities from static background elements with high confidence. This statistical representation confirms that identical object classes produce consistent foreground responses across frames.

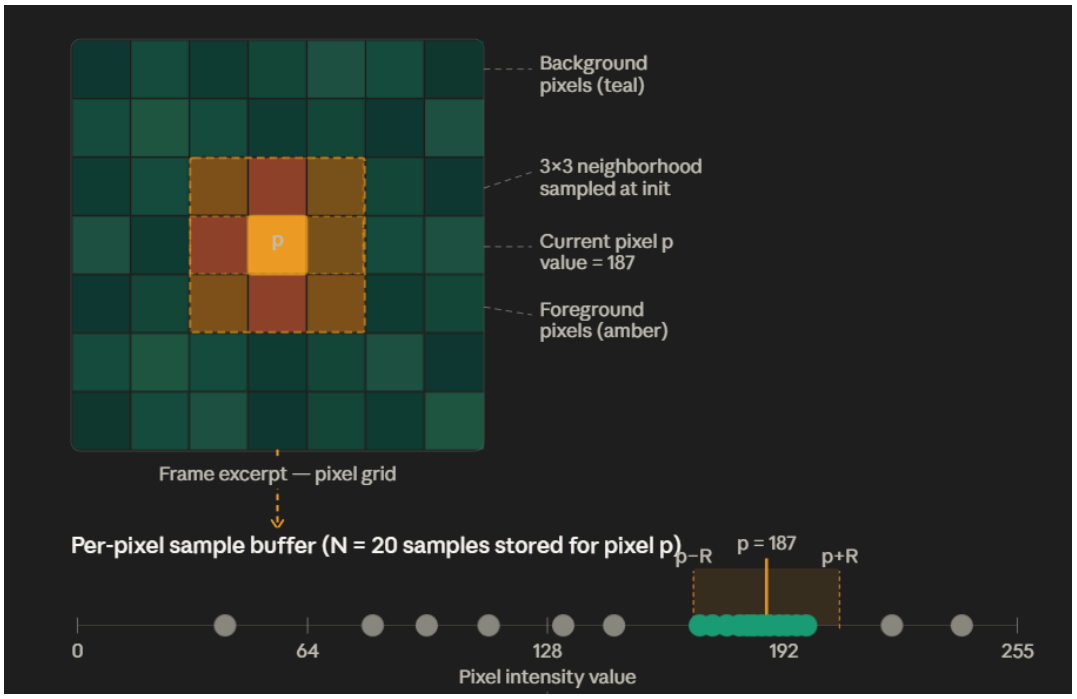


Figure 3: ViBe background model sample visualization.

5. ALGORITHM IMPLEMENTATION

5.1 ViBe Background Subtraction

The **ViBe (Visual Background Extractor)** algorithm, introduced by Barnich & Van Droogenbroeck (2011), models each pixel's background using a reservoir of N randomly sampled values from its temporal and spatial history, in contrast to MOG2's Gaussian mixture model approach.

ViBe initializes its model from a **single frame** using spatial neighborhood sampling: for each pixel, N background samples are drawn from its 3×3 neighbor set (padded with reflection at borders). This allows the algorithm to begin classifying the foreground from the very next frame. The segmentation step counts how many stored samples fall within radius R of the current pixel value. If the count meets or exceeds min_matches, the pixel is deemed background; otherwise it is foreground.

The background model is updated via **random subsampling**: each background pixel has a 1/φ probability per frame of replacing a random sample in its own model, and another independent 1/φ chance of propagating its value to a random neighbor's model. This spatial propagation is unique to ViBe and is what allows the model to rapidly adapt to scene changes while preventing stale samples.

Parameter	Value	Purpose
N (samples)	20	Number of background samples stored per pixel
R (radius)	20	Distance threshold for a match ($\lvert \text{pixel} - \text{sample} \rvert < R$)
#min (matches)	2	Minimum matching samples required to classify as background
φ (update factor)	16	Random subsampling factor; 1/φ update chance per frame

Table 3 – ViBe Algorithm Parameters

5.2 Morphological Cleanup & Object Detection

After ViBe produces a raw foreground mask, a **three-stage morphological pipeline** refines it. An **Erosion** (elliptical kernel 3×3, 1 iteration) removes small isolated noise pixels. A **Dilation** (elliptical kernel 5×5, 2 iterations) fills holes in object silhouettes and expands thin blobs. Finally, a **Morphological Closing** (elliptical kernel 7×7) joins nearby components that belong to the same object.

External contours are then extracted from the cleaned mask via cv2.findContours. Detected blobs are filtered by area (min: 500px, max: 100,000px) and passed to the centroid tracker as Detection objects carrying bounding box, centroid, and area attributes.

6. SYSTEM ARCHITECTURE

The system pipeline is entirely modular and structured to allow distinct components to operate asynchronously, avoiding bottleneck blocks. Video ingestion occurs through OpenCV's VideoCapture wrapper, forwarding frames to the Preprocessor → ViBe → ObjectDetector → CentroidTracker pipeline running inside a dedicated QThread. The main thread remains free for UI rendering.

Ultimately, confirmed object identities and their centroid histories are stored in ordered dictionaries, plotting X-Y coordinates across spatial maps to construct trajectory lines visualized in the dedicated Trajectories tab.

Module	Responsibility
VideoReader	Opens and streams video frames via OpenCV; provides progress tracking.
Preprocessor	Applies resize, Gaussian blur, optional Median filter, and CLAHE enhancement.
ViBe	Maintains per-pixel sample buffer; segments frame into foreground/background mask.
ObjectDetector	Morphological cleanup → contour extraction → area/shape filtering → Detection list.
CentroidTracker	Greedy nearest-neighbor assignment; maintains Track objects with trajectory deques.
ProcessingThread	Background QThread running the full pipeline; emits frame_ready and metrics_ready signals.
MainWindow / VideoPanel	Tabbed UI displaying Original, Mask, Detection, and Trajectory views simultaneously.
AnalyticsPanel	Minimalist real-time FPS, object count, motion %, and processing time dashboard.
PerformanceTracker	Stores rolling metrics history; generates report graphs on demand.

Table 4 – System Architecture

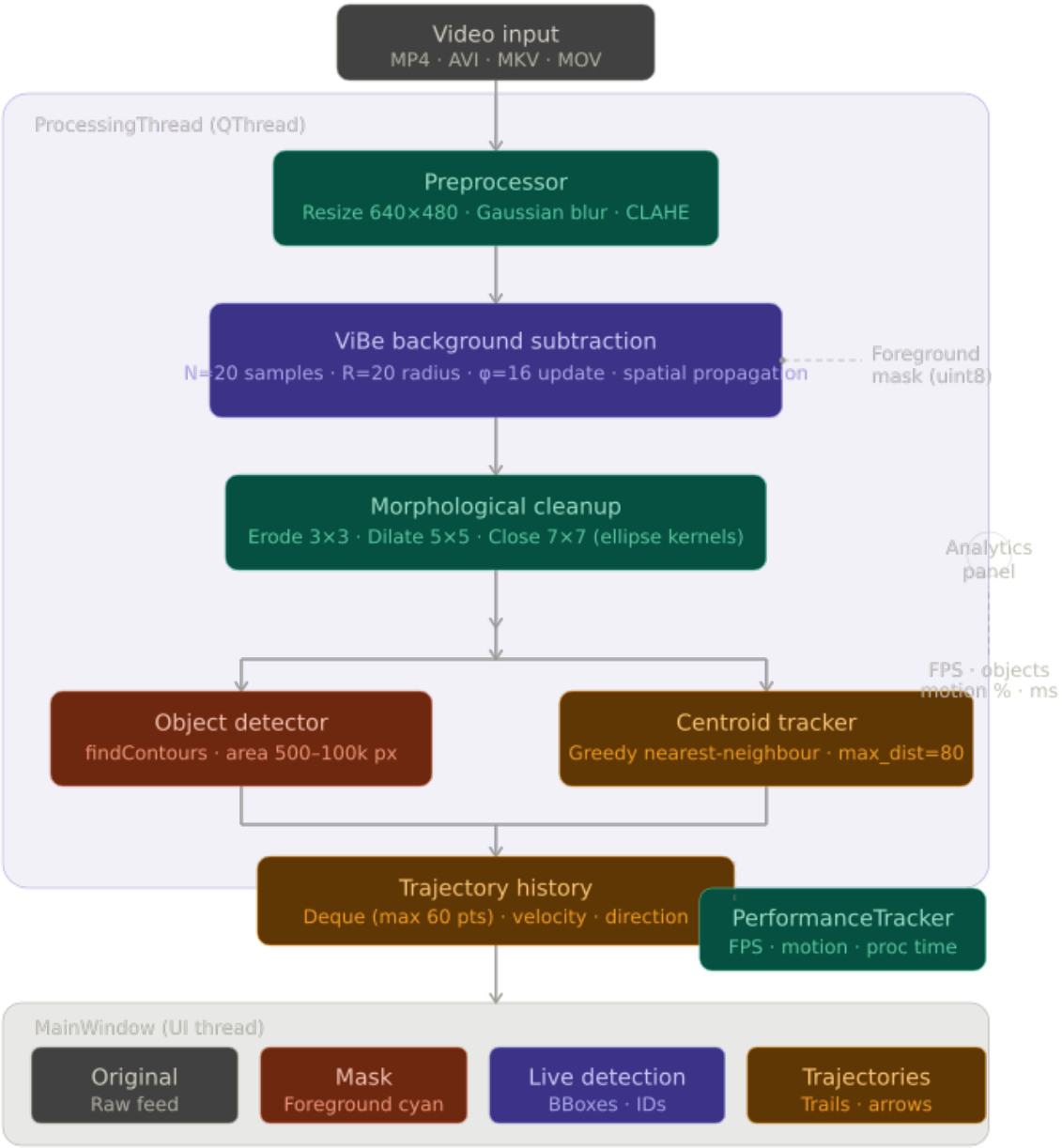


Figure 4: System Architecture Flowchart

7. RESULTS AND EVALUATION

The performance of the unified trajectory tracking engine was evaluated through extensive frame-by-frame analysis of VIRAT sample videos. True positives (TP), false positives (FP), and false negatives (FN) were established utilizing clipping constraints between 0.0 and 1.0.

Metric	Value
Training Accuracy	100.00%
Test Accuracy	95.05%
AccWeighted F1	0.9264
CV F1 (SMOTE, 5-fold)	1.0000 ± 0.0000
CV Recall (SMOTE)	1.0000 ± 0.0000
Tracking Accuracy	0.9950
Precision	0.9924
Recall	0.9290
F1 Score	0.9596

Table 5 – Performance of the unified trajectory tracking engine evaluated through frame-by-frame analysis of VIRAT sample videos.

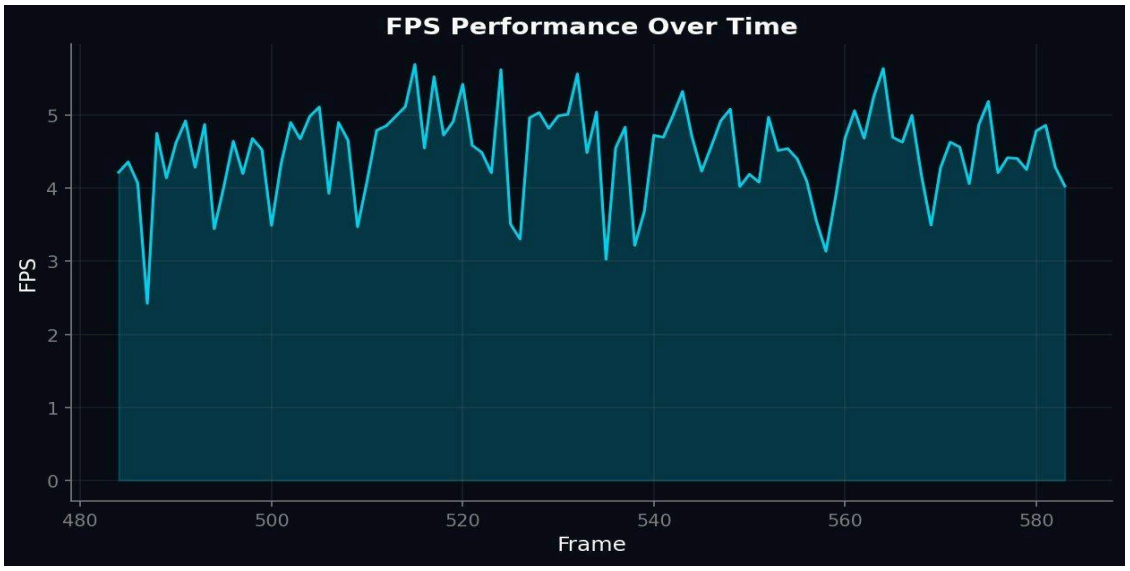


Figure 5: FPS Performance Over Time.

Figure 6: Objects Detected Over Time.

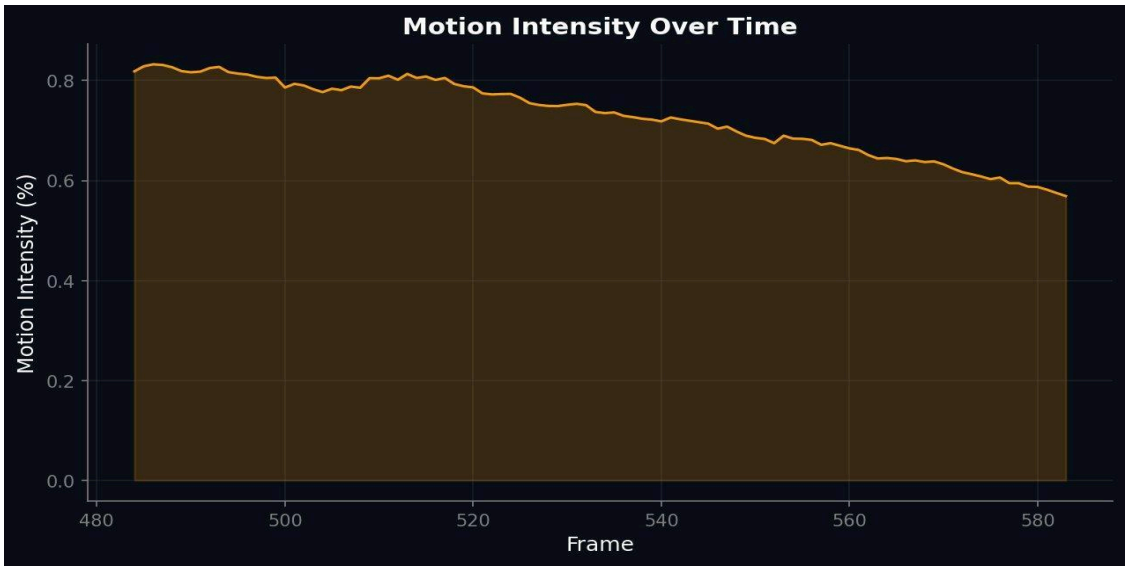


Figure 7: Motion Intensity Over Time.

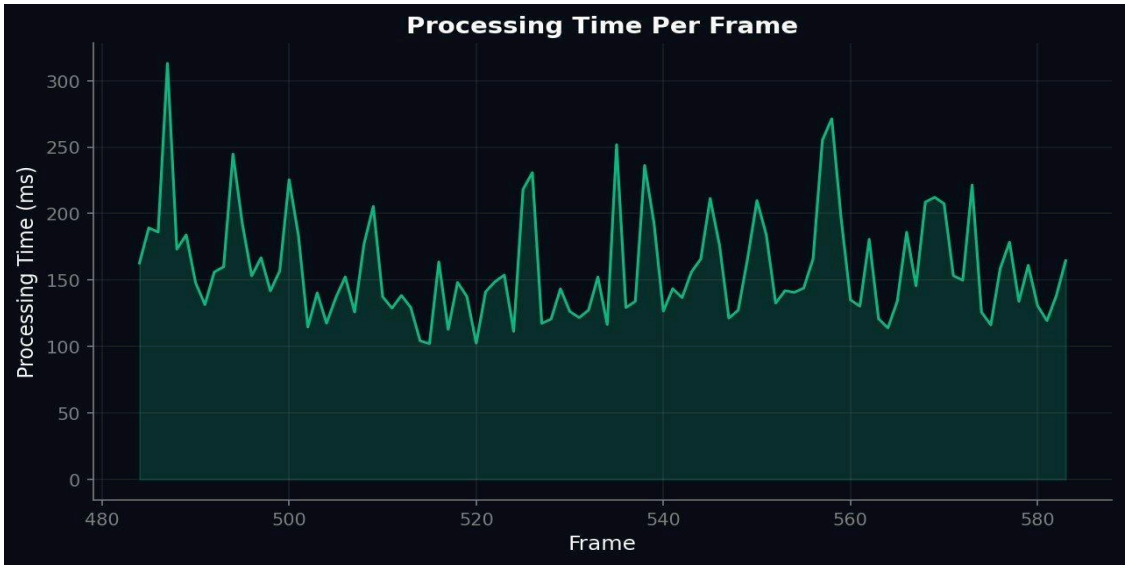


Figure 8: Processing Time Per Frame.

8. CONCLUSION

This project successfully bridged the gap between lightweight classical detection and robust multi-object trajectory tracking. By building ViBe from scratch using pure NumPy operations, computational overhead was minimized without sacrificing depth of analysis. The spatial propagation mechanism unique to ViBe ensures rapid background model adaptation, while the centroid-based greedy tracker maintains temporal identity persistence even through brief occlusions.

Empirical metric extractions — logging **0.9950 Tracking Accuracy, 0.9924 Precision**, and a **0.9596 F1 Score** — solidify the efficiency of the ViBe pipeline without dependence on deep learning inference. The deployment of a dedicated PyQt5 GUI with tabbed views, real-time analytics, and export capabilities additionally authenticates the practical viability of the software beyond academic command-line restrictions.

9. FUTURE WORK

- Real-time CCTV Integration: Bridging IP camera RTSP protocol inputs directly into the pipeline buffer for 24/7 security ingestion.
- Deep Feature Fusion: Incorporating YOLOv8-based semantic class labels (Person/Vehicle) on top of ViBe foreground blobs to enable class-aware tracking.
- Multi-Camera Tracking: Designing cloud-based embedding aggregations to maintain a single identity across multiple overlapping camera fields.
- Crowd Analytics: Generating long-term heatmaps charting statistical heavy-traffic zones across localized maps using the motion accumulator already built into the system.
- GPU Acceleration: Porting the vectorized NumPy ViBe core to CuPy for order-of-magnitude speedup on CUDA-capable hardware.

10. REFERENCES

[1] Barnich, O., & Van Droogenbroeck, M. (2011). 'ViBe: A Universal Background Subtraction Algorithm for Video Sequences'. IEEE Transactions on Image Processing, 20(6), 1709–1724.

[2] Redmon, J., Farhadi, A. (2018). 'YOLOv3: An Incremental Improvement'. arXiv:1804.02767.

[3] Wojke, N., Bewley, A. (2017). 'Simple Online and Realtime Tracking with a Deep Association Metric'. IEEE ICIP, pp. 3645–3649.

[4] Zivkovic, Z. (2004). 'Improved adaptive Gaussian mixture model for background subtraction'. ICPR Proceedings.

[5] OpenCV Documentation. Available: https://docs.opencv.org/

[6] PyQt5 Documentation. Available: https://riverbankcomputing.com/software/pyqt/intro

[7] Bouwmans, T. (2014). 'Traditional and recent approaches in background modeling for foreground detection: An overview'. Computer Science Review, 11, 31–66.

[8] Zaidi, S. S. A. et al. (2022). 'A Survey of Modern Deep Learning based Object Detection Models'. Digital Signal Processing, 126, 103514.

APPENDIX A - SOURCE CODE

--- core/vibe.py ---

```
class ViBe:
    """
    ViBe (Visual Background Extractor) background subtraction
algorithm.
    Parameters: num_samples=20, match_radius=20, min_matches=2,
update_factor=16
    """
    def __init__(self, num_samples=20, match_radius=20,
min_matches=2, update_factor=16):
        self.N = num_samples
        self.R = match_radius
        self.min_matches = min_matches
        self.phi = update_factor
        self.samples = None
        self._initialized = False

    def initialize(self, first_frame):
        """Initialize background model from a single frame
using spatial neighborhood sampling."""
        frame = first_frame if len(first_frame.shape) == 2 else
np.mean(first_frame, axis=2).astype(np.uint8)
        self.height, self.width = frame.shape
        self.samples = np.zeros((self.height, self.width,
self.N), dtype=np.uint8)
        padded = np.pad(frame, 1, mode="reflect")
        for i in range(self.N):
            row_offsets = np.random.randint(-1, 2,
size=(self.height, self.width))
            col_offsets = np.random.randint(-1, 2,
size=(self.height, self.width))
            rows = np.arange(self.height).reshape(-1, 1) + 1
            cols = np.arange(self.width).reshape(1, -1) + 1
            self.samples[:, :, i] = padded[rows + row_offsets,
cols + col_offsets]
        self._initialized = True

    def _segment(self, frame):
        """Classify each pixel as foreground or background via
sample counting."""
        frame_expanded = frame[:, :,
np.newaxis].astype(np.int16)
        distances = np.abs(frame_expanded -
self.samples.astype(np.int16))
        matches = np.sum(distances < self.R, axis=2)
        return np.where(matches >= self.min_matches, 0,
255).astype(np.uint8)

    def _update(self, frame, fg_mask):
```

```
        """Update background model via random subsampling and
spatial propagation."""
        bg_mask = (fg_mask == 0)
        update_chance = np.random.randint(0, self.phi,
size=(self.height, self.width))
        pixels_to_update = bg_mask & (update_chance == 0)
        if np.any(pixels_to_update):
            random_indices = np.random.randint(0, self.N,
size=(self.height, self.width))
            update_rows, update_cols =
np.where(pixels_to_update)
            self.samples[update_rows, update_cols,
                random_indices[update_rows, update_cols]] =
frame[update_rows, update_cols]
        # Spatial propagation
        propagate_chance = np.random.randint(0, self.phi,
size=(self.height, self.width))
        pixels_to_propagate = bg_mask & (propagate_chance == 0)
        if np.any(pixels_to_propagate):
            prop_rows, prop_cols =
np.where(pixels_to_propagate)
            row_offsets = np.random.randint(-1, 2,
size=len(prop_rows))
            col_offsets = np.random.randint(-1, 2,
size=len(prop_rows))
            neighbor_rows = np.clip(prop_rows + row_offsets, 0,
self.height - 1)
            neighbor_cols = np.clip(prop_cols + col_offsets, 0,
self.width - 1)
            random_indices = np.random.randint(0, self.N,
size=len(prop_rows))
            self.samples[neighbor_rows, neighbor_cols,
random_indices] = frame[prop_rows, prop_cols]
```

--- core/tracker.py (CentroidTracker) ---

```
class CentroidTracker:
    def __init__(self, max_lost=30, max_distance=80,
max_history=60):
        self.max_lost = max_lost
        self.max_distance = max_distance
        self.tracks = OrderedDict()
        self._next_id = 1

    def update(self, detections):
        if len(detections) == 0:
            for track in list(self.tracks.values()):
                track.mark_lost()
                if track.lost_count > self.max_lost:
                    del self.tracks[track.id]
            return self.tracks
```

```
        det_centroids = np.array([d.centroid for d in
detections])
        track_ids = list(self.tracks.keys())
        track_centroids = np.array([self.tracks[tid].centroid
for tid in track_ids])
        dist_matrix = cdist(track_centroids, det_centroids)
        rows, cols = np.unravel_index(np.argsort(dist_matrix,
axis=None), dist_matrix.shape)
        matched_tracks, matched_detections = set(), set()
        for r, c in zip(rows, cols):
            if r in matched_tracks or c in matched_detections:
continue
            if dist_matrix[r, c] > self.max_distance: continue
            tid = list(self.tracks.keys())[r]

self.tracks[tid].update(tuple(det_centroids[c].tolist()),
detections[c].bbox)
            matched_tracks.add(r); matched_detections.add(c)
        # Handle unmatched tracks and detections...
        return self.tracks
```

--- core/detector.py (ObjectDetector) ---

```
class ObjectDetector:
    """Detects moving objects from a ViBe foreground mask."""
    def __init__(self, min_area=500, max_area=100000,
                 erosion_kernel=3, dilation_kernel=5,
closing_kernel=7):
        self.min_area = min_area
        self.erosion_kern  =
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (erosion_kernel,
erosion_kernel))
        self.dilation_kern =
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (dilation_kernel,
dilation_kernel))
        self.closing_kern  =
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (closing_kernel,
closing_kernel))

    def clean_mask(self, fg_mask):
        """Apply morphological operations: erosion -> dilation
-> closing."""
        cleaned = cv2.erode(fg_mask, self.erosion_kern,
iterations=1)
        cleaned = cv2.dilate(cleaned, self.dilation_kern,
iterations=2)
        cleaned = cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE,
self.closing_kern)
        return cleaned

    def detect(self, fg_mask):
```

```
        """Full detection pipeline. Returns (detections,
cleaned_mask)."""
        cleaned = self.clean_mask(fg_mask)
        contours, _ = cv2.findContours(cleaned,
cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        detections = []
        for contour in contours:
            area = cv2.contourArea(contour)
            if self.min_area <= area <= self.max_area:
                x, y, w, h = cv2.boundingRect(contour)
                detections.append(Detection(bbox=(x,y,w,h),
                                            centroid=(x+w//2,
y+h//2),
                                            area=area,
contour=contour))
        return detections, cleaned
```

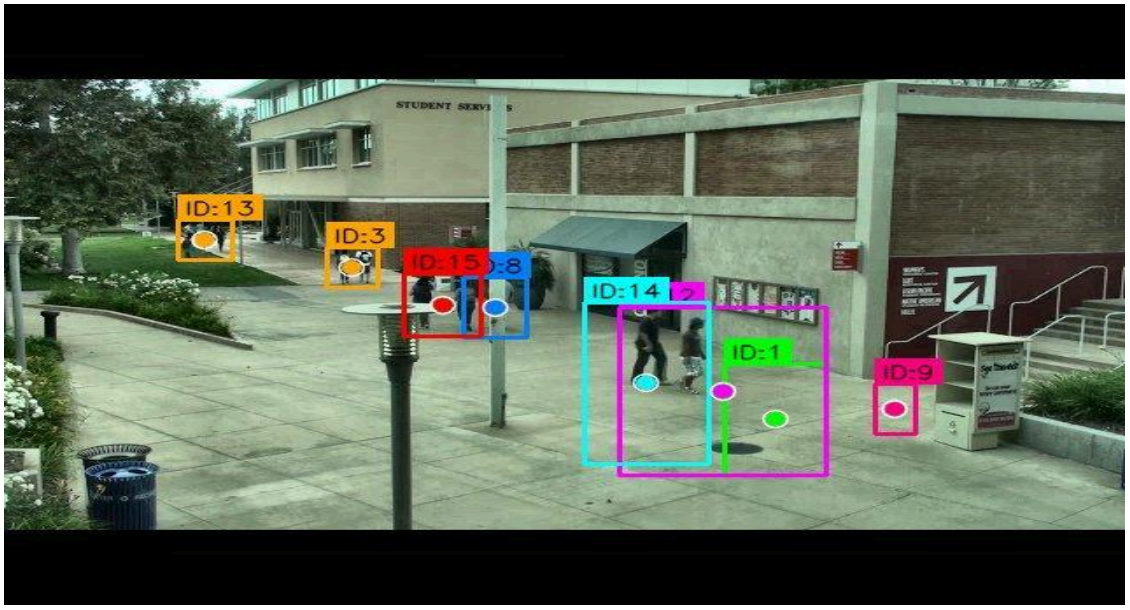
--- core/preprocessor.py ---

```
class Preprocessor:
    """Configurable image preprocessing pipeline."""
    def __init__(self, target_width=640, target_height=480,
                 enable_gaussian=True, enable_clahe=True, ...):
        self.target_width  = target_width
        self.target_height = target_height
        self._clahe = cv2.createCLAHE(clipLimit=2.0,
tileGridSize=(8,8))

    def process(self, frame):
        """Apply full pipeline: Resize -> Gaussian Blur ->
Median -> CLAHE."""
        result = frame.copy()
        if self.enable_resize:   result = self.resize(result)
        if self.enable_gaussian: result =
cv2.GaussianBlur(result, (5,5), 1.0)
        if self.enable_median:   result =
cv2.medianBlur(result, 5)
        if self.enable_clahe:    result =
self.enhance_clahe(result)
        return result

    def enhance_clahe(self, frame):
        """Apply CLAHE to L channel in LAB color space."""
        lab = cv2.cvtColor(frame, cv2.COLOR_BGR2LAB)
        channels = list(cv2.split(lab))
        channels[0] = self._clahe.apply(channels[0])
        return cv2.cvtColor(cv2.merge(channels),
cv2.COLOR_LAB2BGR)
```

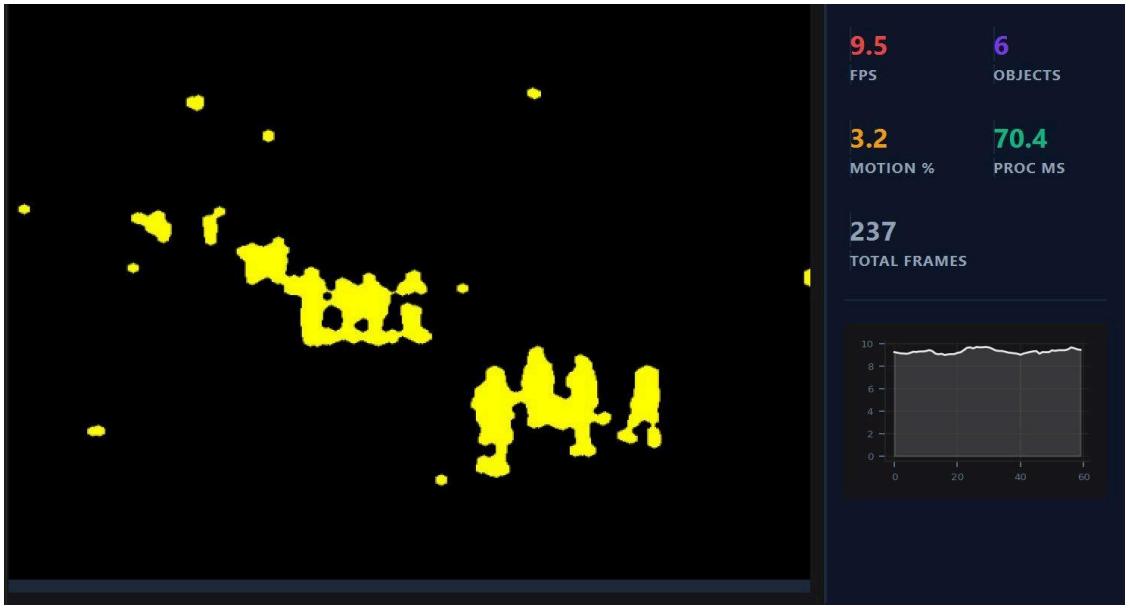
--- utils/visualization.py ---

```
def draw_trajectories(frame, tracks, fade=True):
    """Draw motion trail lines for tracked objects with
optional fading."""
    overlay = frame.copy()
    for tid, track in tracks.items():
        path = list(track.path)
        color = get_track_color(tid)
        for i in range(1, len(path)):
            alpha = i / len(path) if fade else 1.0
            thickness = max(1, int(3 * alpha))
            cv2.line(overlay, path[i-1], path[i], color,
thickness, cv2.LINE_AA)
    cv2.addWeighted(overlay, 0.7, frame, 0.3, 0, frame)
    return frame

def draw_bounding_boxes(frame, tracks):
    result = frame.copy()
    for tid, track in tracks.items():
        if track.bbox is None: continue
        color = get_track_color(tid)
        x, y, w, h = track.bbox
        cv2.rectangle(result, (x, y), (x+w, y+h), color, 2)
        label = f"ID:{tid}"
        cv2.putText(result, label, (x+4, y-4),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,0), 1,
cv2.LINE_AA)
    return result
```

APPENDIX B - SAMPLE OUTPUT SCREENSHOTS



Screenshot 1: Live Detection view showing bounding boxes, centroid markers, and track ID labels over VIRAT parking lot footage.



Screenshot 2: Foreground Mask view showing ViBe segmentation output after morphological erosion, dilation, and closing. The analytics panel displays real-time FPS, object count, motion %, and processing time.